

Mata Kuliah Workshop Pemrograman Framework
LAPORAN E - GOVERNANCE
Sistem Pengajuan Subsidi Pupuk Kabupaten Mojokerto



Dosen Pengampu:
Amma Liesvarastranta Haz, S.Tr.T., M.T.

Disusun Oleh:
Syarifatullah Ceva Efendy (312521002)
Selby Wafi Nurjuan (3125521018)

PROGRAM STUDI D3 TEKNIK INFORMATIKA PSDKU LAMONGAN
DEPARTEMEN TEKNIK INFORMATIKA DAN KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2026

E-Pupuk: Sistem Pengajuan Subsidi Pupuk Kabupaten Mojokerto Berbasis Data Terbuka Pemerintah

Syarifatullah Ceva Efendy*, Selby Wafi Nurjuan[†]

Program Studi D3 Teknik Informatika PSDKU Lamongan

Politeknik Elektronika Negeri Surabaya

Email: *312521002@student.pens.ac.id, [†]3125521018@student.pens.ac.id

Abstract—Pengelolaan dan penyaluran pupuk bersubsidi merupakan aspek krusial dalam mendukung produktivitas sektor pertanian di Kabupaten Mojokerto. Namun, proses rekapitulasi data dan pengajuan manual yang belum terintegrasi sering menimbulkan kendala seperti keterlambatan informasi, kesalahan pencatatan, serta ketidaksesuaian alokasi kuota. Penelitian ini mengusulkan E-Pupuk, sebuah sistem informasi berbasis web yang mengintegrasikan pendataan petani, pemeriksaan kuota RDKK, verifikasi oleh Petugas Penyuluh Lapangan (PPL), pencatatan realisasi distribusi, hingga pemantauan oleh Dinas Pertanian. Dengan memanfaatkan data terbuka pemerintah (open data) sebagai acuan awal, E-Pupuk meningkatkan efisiensi administrasi, transparansi informasi, serta akurasi penyaluran pupuk bersubsidi agar lebih tepat sasaran.

Index Terms—e-Governance, E-Pupuk, Subsidi Pupuk, Open Data, RDKK, Kabupaten Mojokerto.

I. PENDAHULUAN

A. Latar Belakang

Perkembangan teknologi informasi dan komunikasi mendorong transformasi tata kelola pemerintahan menuju *e-Governance*. Perubahan ini menuntut pelayanan publik yang lebih transparan, efisien, dan akuntabel melalui pemanfaatan teknologi serta pengelolaan data secara terintegrasi. Salah satu penerapannya adalah pemanfaatan data terbuka pemerintah (*open data*) sebagai sumber data yang dapat mendukung proses pelayanan publik secara aktif [1].

Dalam sektor pertanian, pemerintah Kabupaten Mojokerto menyalurkan program pupuk bersubsidi untuk membantu petani menjaga produktivitas dan mengurangi biaya produksi. Alokasi pupuk subsidi mengacu pada Rencana Definitif Kebutuhan Kelompok (RDKK) yang disusun oleh kelompok tani dan diteruskan kepada Dinas Pertanian. Namun, proses pengelolaan dan rekapitulasi data yang masih belum terintegrasi dapat menyebabkan keterlambatan penyampaian informasi, kesalahan pencatatan, serta kesulitan dalam melakukan verifikasi pengajuan pupuk [2].

Kondisi tersebut juga menyebabkan petani kesulitan memperoleh informasi mengenai sisa kuota kelompok dan status pengajuan mereka. Di sisi lain, Petugas Penyuluh Lapangan (PPL) membutuhkan waktu dalam melakukan verifikasi pengajuan, sedangkan pihak Dinas Pertanian memerlukan sistem

yang dapat membantu memantau realisasi penyaluran pupuk secara lebih cepat dan terstruktur. Oleh karena itu, diperlukan sistem yang mampu mengintegrasikan proses pengajuan, verifikasi, distribusi, serta pemantauan data pupuk bersubsidi [3].

Sebagai solusi, dikembangkan E-Pupuk – Sistem Pengajuan Subsidi Pupuk Kabupaten Mojokerto, yaitu sistem berbasis web yang mengintegrasikan pendataan petani dan lahan, pengajuan pupuk, pemeriksaan kuota RDKK, verifikasi oleh PPL, pencatatan realisasi distribusi, serta penyusunan laporan. Sistem ini diharapkan dapat meningkatkan efisiensi proses administrasi, mengurangi kesalahan data, meningkatkan transparansi informasi, dan membantu mewujudkan distribusi pupuk bersubsidi yang lebih tepat sasaran di Kabupaten Mojokerto [4].

B. Rumusan Masalah

- 1) Bagaimana mengelola data petani dan kelompok tani penerima pupuk subsidi secara terpusat?
- 2) Bagaimana menentukan kewajaran kuota pengajuan pupuk berdasarkan data RDKK?
- 3) Bagaimana membantu petugas penyuluh melakukan verifikasi pengajuan secara cepat dan objektif?
- 4) Bagaimana petani dapat mengetahui status pengajuan dan distribusi pupuknya?
- 5) Bagaimana Dinas Pertanian dapat memantau realisasi distribusi pupuk subsidi secara cepat dan terstruktur?

C. Tujuan

- 1) Membuat sistem pengelolaan data petani dan kelompok tani secara terintegrasi.
- 2) Membantu proses verifikasi pengajuan berdasarkan data RDKK dan luas lahan.
- 3) Mempermudah petani memperoleh informasi status pengajuan dan distribusi pupuk.
- 4) Menyediakan dashboard monitoring dan pelaporan bagi Dinas Pertanian.

ACKNOWLEDGMENT

Penulis menyampaikan terima kasih kepada Dosen Pengampu Mata Kuliah Workshop Pemrograman Framework, Bapak Amma Liesvarastranta Haz, S.Tr.T., M.T., atas bimbingan dan arahan selama penyusunan laporan ini, serta Politeknik Elektronika Negeri Surabaya.

REFERENCES

- [1] R. Heeks, "Implementing and Managing e-Government: An International Text," SAGE Publications, 2006.
- [2] Kementerian Pertanian Republik Indonesia, "Pedoman Pengelolaan Sistem Elektronik Rencana Definitif Kebutuhan Kelompok (e-RDKK)," Jakarta, 2021.
- [3] Dinas Pertanian Kabupaten Mojokerto, "Laporan Evaluasi Penyaluran Pupuk Bersubsidi Kabupaten Mojokerto," 2024.
- [4] A. S. Ahmad, "Pemanfaatan Open Government Data untuk Transparansi Layanan Publik di Sektor Pertanian," *Jurnal Teknologi dan Governance*, vol. 8, no. 1, pp. 45–56, 2023.

II. SUMBER KODE

A. Frontend — App.jsx

Listing 1. frontend/src/App.jsx

```
1 import React, { useEffect } from 'react';
2 import { BrowserRouter, Routes, Route, Navigate, useNavigate } from 'react-router-dom';
3 import { Toaster } from 'react-hot-toast';
4 import { useAuthStore } from './store/authStore';
5
6 // Components
7 import Navbar from './components/Navbar';
8
9 // Pages
10 import Landing from './pages/Landing';
11 import Register from './pages/Register';
12 import Login from './pages/Login';
13 import PetaniDashboard from './pages/PetaniDashboard';
14 import AdminDashboard from './pages/AdminDashboard';
15 import PPLDashboard from './pages/PPLDashboard';
16 import QRScannerKiosk from './pages/QRScannerKiosk';
17
18 // Protected Route Guard
19 function ProtectedRoute({ children, allowedRoles }) {
20   const { isAuthenticated, user, isLoading } = useAuthStore();
21
22   if (isLoading) {
23     return (
24       <div className="min-h-screen flex items-center justify-center bg-slate-50">
25         <div className="w-8 h-8 border-4 border-emerald-600 border-t-transparent rounded-full animate-spin" />
26       </div>
27     );
28   }
29
30   if (!isAuthenticated || !user) {
31     return <Navigate to="/login" replace />;
32   }
33
34   if (allowedRoles && !allowedRoles.includes(user.role)) {
35     // Redirect to their respective dashboard
36     if (user.role === 'ADMIN') return <Navigate to="/admin" replace />;
37     if (user.role === 'PPL') return <Navigate to="/ppl" replace />;
38     return <Navigate to="/petani" replace />;
39   }
40
41   return children;
42 }
43
44 // App Layout with Navbar
45 function DashboardLayout({ children }) {
46   return (
47     <div className="min-h-screen bg-slate-50 flex flex-col font-sans">
48       <Navbar />
49       <main className="flex-1 max-w-7xl mx-auto w-full px-4 sm:px-6 lg:px-8 pt-6">
50         {children}
51       </main>
52     </div>
53   );
54 }
55
56 // Root redirector
57 function RootRedirect() {
58   const { isAuthenticated, user, isLoading } = useAuthStore();
59
60   if (isLoading) {
61     return (
62       <div className="min-h-screen flex items-center justify-center bg-slate-50">
63         <div className="w-8 h-8 border-4 border-emerald-600 border-t-transparent rounded-full animate-spin" />
64       </div>
65     );
66   }
67
68   if (!isAuthenticated || !user) {
69     return <Navigate to="/login" replace />;
70   }
71
72   if (user.role === 'ADMIN') return <Navigate to="/admin" replace />;
73   if (user.role === 'PPL') return <Navigate to="/ppl" replace />;
74   return <Navigate to="/petani" replace />;
75 }
76
77 export default function App() {
```

```

78  const { initAuth } = useAuthStore();
79
80  useEffect(() => {
81    initAuth();
82  }, [initAuth]);
83
84  return (
85    <BrowserRouter>
86      <Toaster position="top-right" />
87      <Routes>
88        { /* Landing Page */ }
89        <Route path="/" element={<Landing />} />
90
91        { /* Public Login & Register Routes */ }
92        <Route path="/login" element={<Login />} />
93        <Route path="/login/:portalRole" element={<Login />} />
94        <Route path="/register" element={<Register />} />
95
96        { /* Kiosk Scanner Route */ }
97        <Route path="/kiosk-scanner" element={<QRScannerKiosk />} />
98
99        { /* Petani Dashboard Route */ }
100       <Route
101         path="/petani"
102         element={
103           <ProtectedRoute allowedRoles={['PETANI', 'ADMIN']}>
104             <DashboardLayout>
105               <PetaniDashboard />
106             </DashboardLayout>
107           </ProtectedRoute>
108         }
109       />
110
111       { /* Admin Dashboard Route */ }
112       <Route
113         path="/admin"
114         element={
115           <ProtectedRoute allowedRoles={['ADMIN']}>
116             <DashboardLayout>
117               <AdminDashboard />
118             </DashboardLayout>
119           </ProtectedRoute>
120         }
121       />
122
123       { /* PPL Dashboard Route */ }
124       <Route
125         path="/ppl"
126         element={
127           <ProtectedRoute allowedRoles={['PPL', 'ADMIN']}>
128             <DashboardLayout>
129               <PPLDashboard />
130             </DashboardLayout>
131           </ProtectedRoute>
132         }
133       />
134
135       { /* Fallback Redirect */ }
136       <Route path="*" element={<Navigate to="/" replace />} />
137     </Routes>
138   </BrowserRouter>
139 );
140 }

```

Listing 2. backend/app/main.py

```
1 from fastapi import FastAPI
2 from fastapi.middleware.cors import CORSMiddleware
3 from fastapi.staticfiles import StaticFiles
4 from app.core.config import settings
5 from app.routers import (
6     auth,
7     farmers,
8     lands,
9     applications,
10    admin,
11    ppl,
12    distributions,
13    files
14 )
15
16 app = FastAPI(
17     title=settings.PROJECT_NAME,
18     description="Sistem Verifikasi & Distribusi Pupuk Bersubsidi Kabupaten Mojokerto",
19     version="1.0.0"
20 )
21
22 # CORS Middleware
23 app.add_middleware(
24     CORSMiddleware,
25     allow_origins=["*"],
26     allow_credentials=True,
27     allow_methods=["*"],
28     allow_headers=["*"],
29 )
30
31 # Include Routers
32 app.include_router(auth.router, prefix=settings.API_V1_STR)
33 app.include_router(farmers.router)
34 app.include_router(lands.router)
35 app.include_router(applications.router)
36 app.include_router(admin.router)
37 app.include_router(ppl.router)
38 app.include_router(distributions.router)
39 app.include_router(files.router)
40
41 @app.get("/")
42 def root():
43     return {
44         "app": settings.PROJECT_NAME,
45         "status": "online",
46         "docs_url": "/docs"
47     }
48
49 @app.get("/api/health")
50 def health_check():
51     return {"status": "ok", "service": "epupuk-backend"}
```

Listing 3. backend/app/routers/auth.py

```

1 from fastapi import APIRouter, Depends, HTTPException, status
2 from sqlalchemy.orm import Session
3 from app.core.database import get_db
4 from app.core.security import verify_password, create_access_token
5 from app.core.dependencies import get_current_user, log_audit_action
6 from app.models.models import User, Farmer
7 from app.schemas.schemas import LoginRequest, Token, UserResponse
8
9 router = APIRouter(prefix="/auth", tags=["Authentication"])
10
11 @router.post("/login", response_model=Token)
12 def login(request: LoginRequest, db: Session = Depends(get_db)):
13     user = db.query(User).filter(User.username == request.username).first()
14     if not user or not verify_password(request.password, user.password_hash):
15         raise HTTPException(
16             status_code=status.HTTP_401_UNAUTHORIZED,
17             detail="Username atau kata sandi tidak valid."
18         )
19     if not user.is_active:
20         raise HTTPException(
21             status_code=status.HTTP_403_FORBIDDEN,
22             detail="Akun ini telah dinonaktifkan."
23         )
24
25     # If role is PETANI, retrieve farmer profile id and name
26     farmer_id = None
27     display_name = user.username
28     if user.role == "PETANI":
29         farmer = db.query(Farmer).filter(Farmer.user_id == user.id).first()
30         if farmer:
31             farmer_id = farmer.id
32             display_name = farmer.nama
33
34     token = create_access_token(data={
35         "user_id": user.id,
36         "username": user.username,
37         "role": user.role
38     })
39
40     log_audit_action(db, user_id=user.id, action="LOGIN", resource="auth", details=f"User {user.username} logged in")
41
42     return Token(
43         access_token=token,
44         token_type="bearer",
45         role=user.role,
46         user_id=user.id,
47         username=user.username,
48         nama=display_name,
49         farmer_id=farmer_id
50     )
51
52 @router.get("/me")
53 def get_current_profile(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)):
54     farmer_id = None
55     farmer_data = None
56     if current_user.role == "PETANI":
57         farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
58         if farmer:
59             farmer_id = farmer.id
60             farmer_data = {
61                 "id": farmer.id,
62                 "nama": farmer.nama,
63                 "nik": farmer.nik,
64                 "alamat": farmer.alamat,
65                 "kontak": farmer.kontak,
66                 "foto_ktp_url": farmer.foto_ktp_url
67             }
68
69     return {
70         "id": current_user.id,
71         "username": current_user.username,
72         "email": current_user.email,
73         "role": current_user.role,
74         "farmer_id": farmer_id,
75         "farmer": farmer_data
76     }
77
78 @router.post("/logout")
79 def logout(current_user: User = Depends(get_current_user), db: Session = Depends(get_db)):

```

```
80 log_audit_action(db, user_id=current_user.id, action="LOGOUT", resource="auth")
81 return {"message": "Berhasil keluar dari sistem"}
```

Listing 4. backend/app/routers/applications.py

```

1 from typing import List, Optional
2 from decimal import Decimal
3 from fastapi import APIRouter, Depends, HTTPException, status
4 from sqlalchemy.orm import Session, joinedload
5 from app.core.database import get_db
6 from app.core.dependencies import get_current_user, log_audit_action
7 from app.models.models import (
8     User, Farmer, Land, Application, ApplicationStatus,
9     Fertilizer, Notification, FieldSurvey
10 )
11 from app.schemas.schemas import (
12     ApplicationCreate, ApplicationResponse, DashboardStatsPetani
13 )
14
15 router = APIRouter(prefix="/api/applications", tags=["Applications"])
16
17 @router.get("", response_model=List[ApplicationResponse])
18 def get_applications(
19     status_filter: Optional[str] = None,
20     farmer_id: Optional[int] = None,
21     current_user: User = Depends(get_current_user),
22     db: Session = Depends(get_db)
23 ):
24     query = db.query(Application).options(
25         joinedload(Application.farmer),
26         joinedload(Application.land),
27         joinedload(Application.fertilizer),
28         joinedload(Application.survey)
29     )
30
31     if current_user.role == "PETANI":
32         farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
33         if not farmer:
34             return []
35         query = query.filter(Application.farmer_id == farmer.id)
36     elif current_user.role == "PPL":
37         query = query.filter(Application.assigned_ppl_id == current_user.id)
38     elif farmer_id:
39         query = query.filter(Application.farmer_id == farmer_id)
40
41     if status_filter:
42         query = query.filter(Application.status == status_filter)
43
44     query = query.order_by(Application.created_at.desc())
45     return query.all()
46
47 @router.get("/stats/petani", response_model=DashboardStatsPetani)
48 def get_petani_stats(
49     current_user: User = Depends(get_current_user),
50     db: Session = Depends(get_db)
51 ):
52     if current_user.role != "PETANI":
53         raise HTTPException(status_code=403, detail="Hanya petani yang dapat mengakses data ini")
54
55     farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
56     if not farmer:
57         return DashboardStatsPetani(
58             total_pengajuan=0,
59             aktif=0,
60             disetujui=0,
61             tersalurkan=0,
62             total_lahan=0,
63             kuota_tersedia_urea=Decimal("500"),
64             kuota_tersedia_npk=Decimal("400")
65         )
66
67     apps = db.query(Application).filter(Application.farmer_id == farmer.id).all()
68     lands_count = db.query(Land).filter(Land.farmer_id == farmer.id).count()
69
70     total_pengajuan = len(apps)
71     aktif = sum(1 for a in apps if a.status in [
72         ApplicationStatus.DIAJUKAN,
73         ApplicationStatus.MENUNGGU_VERIFIKASI_BERKAS,
74         ApplicationStatus.PERLU_PERBAIKAN_BERKAS,
75         ApplicationStatus.BERKAS_TERVERIFIKASI,
76         ApplicationStatus.DITUGASKAN_KE_PPL,
77         ApplicationStatus.SURVEI_LAPANGAN,
78         ApplicationStatus.MENUNGGU_PERSETUJUAN_AKHIR
79     ])

```

```

80     disetujui = sum(1 for a in apps if a.status in [ApplicationStatus.DISETUIJUI, ApplicationStatus.DIJADWALKAN_DISTRIBUSI
81     ])
82     tersalurkan = sum(1 for a in apps if a.status == ApplicationStatus.TERSALURKAN)
83
84     return DashboardStatsPetani(
85         total_pengajuan=total_pengajuan,
86         aktif=aktif,
87         disetujui=disetujui,
88         tersalurkan=tersalurkan,
89         total_lahan=lands_count,
90         kuota_tersedia_urea=Decimal("500"),
91         kuota_tersedia_npk=Decimal("400")
92     )
93
94 @router.get("/{id}", response_model=ApplicationResponse)
95 def get_application_detail(
96     id: int,
97     current_user: User = Depends(get_current_user),
98     db: Session = Depends(get_db)
99 ):
100     app = db.query(Application).options(
101         joinedload(Application.farmer),
102         joinedload(Application.land),
103         joinedload(Application.fertilizer),
104         joinedload(Application.survey)
105     ).filter(Application.id == id).first()
106
107     if not app:
108         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
109
110     if current_user.role == "PETANI":
111         farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
112         if not farmer or app.farmer_id != farmer.id:
113             raise HTTPException(status_code=403, detail="Akses pengajuan ditolak")
114
115     return app
116
117 @router.post("", response_model=ApplicationResponse)
118 def create_application(
119     data: ApplicationCreate,
120     current_user: User = Depends(get_current_user),
121     db: Session = Depends(get_db)
122 ):
123     farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
124     if not farmer:
125         raise HTTPException(status_code=400, detail="Profil petani tidak ditemukan")
126
127     land = db.query(Land).filter(Land.id == data.land_id, Land.farmer_id == farmer.id).first()
128     if not land:
129         raise HTTPException(status_code=400, detail="Lahan tidak valid atau bukan milik Anda")
130
131     fertilizer = db.query(Fertilizer).filter(Fertilizer.id == data.fertilizer_id).first()
132     if not fertilizer:
133         raise HTTPException(status_code=400, detail="Jenis pupuk tidak valid")
134
135     # Snapshot photos and coordinates
136     new_app = Application(
137         farmer_id=farmer.id,
138         land_id=land.id,
139         fertilizer_id=fertilizer.id,
140         jumlah_diajukan=data.jumlah_diajukan,
141         kuota_maksimal=Decimal("500.00"),
142         status=ApplicationStatus.MENUNGGU_VERIFIKASI_BERKAS,
143         foto_ktp_snapshot_url=farmer.foto_ktp_url,
144         foto_lahan_snapshot_url=land.foto_lahan_url,
145         alamat_lahan=data.alamat_lahan or land.alamat_lahan,
146         latitude=data.latitude or land.latitude,
147         longitude=data.longitude or land.longitude
148     )
149     db.add(new_app)
150     db.commit()
151     db.refresh(new_app)
152
153     # Notification for farmer
154     notif = Notification(
155         user_id=current_user.id,
156         judul="Pengajuan Subsidi Terkirim",
157         pesan=f"Pengajuan subsidi pupuk {fertilizer.nama_pupuk} sebesar {data.jumlah_diajukan} kg telah berhasil diajukan dan sedang menunggu verifikasi berkas oleh Admin.",
158         tipe="INFO"
159     )
160     db.add(notif)
161     db.commit()

```

```

161     log_audit_action(
162         db,
163         user_id=current_user.id,
164         action="CREATE_APPLICATION",
165         resource="applications",
166         resource_id=new_app.id,
167         details=f"Created application #{new_app.id} for {data.jumlah_diajukan} kg {fertilizer.nama_pupuk}"
168     )
169
170     return new_app
171
172
173 @router.put("/{id}/revise-docs", response_model=ApplicationResponse)
174 def revise_application_docs(
175     id: int,
176     foto_ktp_url: Optional[str] = None,
177     foto_lahan_url: Optional[str] = None,
178     current_user: User = Depends(get_current_user),
179     db: Session = Depends(get_db)
180 ):
181     app = db.query(Application).filter(Application.id == id).first()
182     if not app:
183         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
184
185     farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
186     if not farmer or app.farmer_id != farmer.id:
187         raise HTTPException(status_code=403, detail="Akses ditolak")
188
189     if app.status != ApplicationStatus.PERLU_PERBAIKAN_BERKAS:
190         raise HTTPException(status_code=400, detail="Pengajuan tidak dalam status perlu perbaikan berkas")
191
192     if foto_ktp_url:
193         app.foto_ktp_snapshot_url = foto_ktp_url
194         farmer.foto_ktp_url = foto_ktp_url
195     if foto_lahan_url:
196         app.foto_lahan_snapshot_url = foto_lahan_url
197
198     app.status = ApplicationStatus.MENUNGGU_VERIFIKASI_BERKAS
199     db.commit()
200     db.refresh(app)
201
202     # Notification
203     notif = Notification(
204         user_id=current_user.id,
205         judul="Berkas Perbaikan Terkirim",
206         pesan=f"Perbaikan berkas untuk pengajuan #{app.id} telah dikirim ulang ke Admin.",
207         tipe="SUCCESS"
208     )
209     db.add(notif)
210     db.commit()
211
212     return app
213
214 @router.put("/{id}/cancel")
215 def cancel_application(
216     id: int,
217     current_user: User = Depends(get_current_user),
218     db: Session = Depends(get_db)
219 ):
220     app = db.query(Application).filter(Application.id == id).first()
221     if not app:
222         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
223
224     farmer = db.query(Farmer).filter(Farmer.user_id == current_user.id).first()
225     if not farmer or app.farmer_id != farmer.id:
226         raise HTTPException(status_code=403, detail="Akses ditolak")
227
228     if app.status not in [ApplicationStatus.DIAJUKAN, ApplicationStatus.MENUNGGU_VERIFIKASI_BERKAS]:
229         raise HTTPException(status_code=400, detail="Pengajuan tidak dapat dibatalkan pada tahap ini")
230
231     app.status = ApplicationStatus.DITOLAK_BERKAS
232     app.catatan_admin_berkas = "Dibatalkan oleh pemohon"
233     db.commit()
234     return {"message": "Pengajuan berhasil dibatalkan"}

```

Listing 5. backend/app/routers/ppl.py

```

1 from typing import List, Optional
2 from fastapi import APIRouter, Depends, HTTPException, status
3 from sqlalchemy.orm import Session, joinedload
4 from app.core.database import get_db
5 from app.core.dependencies import get_current_user, require_role, log_audit_action
6 from app.models.models import (
7     User, Application, ApplicationStatus, FieldSurvey, Notification
8 )
9 from app.schemas.schemas import (
10     ApplicationResponse, FieldSurveyCreate, FieldSurveyResponse
11 )
12
13 router = APIRouter(prefix="/api/ppl", tags=["PPL Field Survey"])
14
15 @router.get("/assigned-tasks", response_model=List[ApplicationResponse])
16 def get_assigned_tasks(
17     current_user: User = Depends(require_role("PPL", "ADMIN")),
18     db: Session = Depends(get_db)
19 ):
20     query = db.query(Application).options(
21         joinedload(Application.farmer),
22         joinedload(Application.land),
23         joinedload(Application.fertilizer),
24         joinedload(Application.survey)
25     )
26     if current_user.role == "PPL":
27         query = query.filter(Application.assigned_ppl_id == current_user.id)
28
29     query = query.filter(
30         Application.status.in_(
31             ApplicationStatus.DITUGASKAN_KE_PPL,
32             ApplicationStatus.SURVEI_LAPANGAN,
33             ApplicationStatus.MENUNGGU_PERSETUJUAN_AKHIR
34         )
35     )
36     return query.order_by(Application.created_at.desc()).all()
37
38 @router.post("/applications/{id}/start-survey")
39 def start_survey(
40     id: int,
41     current_user: User = Depends(require_role("PPL", "ADMIN")),
42     db: Session = Depends(get_db)
43 ):
44     app = db.query(Application).filter(Application.id == id).first()
45     if not app:
46         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
47
48     if current_user.role == "PPL" and app.assigned_ppl_id != current_user.id:
49         raise HTTPException(status_code=403, detail="Tugas ini bukan untuk Anda")
50
51     app.status = ApplicationStatus.SURVEI_LAPANGAN
52     db.commit()
53     db.refresh(app)
54     return {"message": "Status survei lapangan dimulai", "status": app.status}
55
56 @router.post("/applications/{id}/submit-survey", response_model=FieldSurveyResponse)
57 def submit_survey(
58     id: int,
59     data: FieldSurveyCreate,
60     current_user: User = Depends(require_role("PPL", "ADMIN")),
61     db: Session = Depends(get_db)
62 ):
63     app = db.query(Application).filter(Application.id == id).first()
64     if not app:
65         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
66
67     if current_user.role == "PPL" and app.assigned_ppl_id != current_user.id:
68         raise HTTPException(status_code=403, detail="Tugas ini bukan untuk Anda")
69
70     # Upsert field survey
71     survey = db.query(FieldSurvey).filter(FieldSurvey.application_id == app.id).first()
72     if not survey:
73         survey = FieldSurvey(
74             application_id=app.id,
75             ppl_id=current_user.id,
76             kondisi_fisik_lahan=data.kondisi_fisik_lahan,
77             keterangan_fisik_lahan=data.keterangan_fisik_lahan,
78             kondisi_tanaman=data.kondisi_tanaman,
79             keterangan_tanaman=data.keterangan_tanaman,

```

```

80         luas_lahan_aktual_m2=data.luas_lahan_aktual_m2,
81         foto_survei_urls=data.foto_survei_urls,
82         catatan_ppl=data.catatan_ppl,
83         rekomendasi=data.rekomendasi
84     )
85     db.add(survey)
86 else:
87     survey.kondisi_fisik_lahan = data.kondisi_fisik_lahan
88     survey.keterangan_fisik_lahan = data.keterangan_fisik_lahan
89     survey.kondisi_tanaman = data.kondisi_tanaman
90     survey.keterangan_tanaman = data.keterangan_tanaman
91     survey.luas_lahan_aktual_m2 = data.luas_lahan_aktual_m2
92     survey.foto_survei_urls = data.foto_survei_urls
93     survey.catatan_ppl = data.catatan_ppl
94     survey.rekomendasi = data.rekomendasi
95
96 # Advance application status to MENUNGGU_PERSETUJUAN_AKHIR
97 app.status = ApplicationStatus.MENUNGGU_PERSETUJUAN_AKHIR
98
99 # Notify Admin and Farmer
100 admins = db.query(User).filter(User.role == "ADMIN").all()
101 for adm in admins:
102     db.add(Notification(
103         user_id=adm.id,
104         judul="Hasil Survei Lapangan Diserahkan",
105         pesan=f"PPL {current_user.username} telah menyelesaikan survei lapangan untuk pengajuan #{app.id}. Rekomendasi
: {data.rekomendasi}. Menunggu persetujuan akhir Anda.",
106         tipe="INFO"
107     ))
108
109 if app.farmer and app.farmer.user_id:
110     db.add(Notification(
111         user_id=app.farmer.user_id,
112         judul="Survei Lapangan Telah Selesai",
113         pesan=f"Petugas PPL telah menyelesaikan verifikasi fisik lahan Anda. Pengajuan sedang menunggu persetujuan
akhir dari Admin Dinas Pertanian.",
114         tipe="INFO"
115     ))
116
117 db.commit()
118 db.refresh(survey)
119
120 log_audit_action(
121     db,
122     user_id=current_user.id,
123     action="SUBMIT_FIELD_SURVEY",
124     resource="field_surveys",
125     resource_id=survey.id,
126     details=f"Survey submitted for app #{app.id}: {data.rekomendasi}"
127 )
128
129 resp = FieldSurveyResponse.from_orm(survey)
130 resp.ppl_nama = current_user.username
131 return resp

```

Listing 6. backend/app/routers/admin.py

```

1 import datetime
2 from decimal import Decimal
3 from typing import List, Optional
4 from fastapi import APIRouter, Depends, HTTPException, status
5 from sqlalchemy.orm import Session
6 from sqlalchemy import func
7 from app.core.database import get_db
8 from app.core.dependencies import get_current_user, require_role, log_audit_action
9 from app.models.models import (
10     User, Farmer, Land, Application, ApplicationStatus,
11     Distribution, Notification, AuditLog, FieldSurvey
12 )
13 from app.schemas.schemas import (
14     DashboardStatsAdmin, ApplicationResponse,
15     ApplicationAdminVerifyRequest, ApplicationAssignPPLRequest,
16     ApplicationFinalApproveRequest, UserResponse, AuditLogResponse
17 )
18
19 router = APIRouter(prefix="/api/admin", tags=["Admin Management"])
20
21 @router.get("/stats", response_model=DashboardStatsAdmin)
22 def get_admin_stats(
23     current_user: User = Depends(require_role("ADMIN")),
24     db: Session = Depends(get_db)
25 ):
26     total_pengajuan = db.query(Application).count()
27     menunggu_verifikasi_berkas = db.query(Application).filter(
28         Application.status.in_([ApplicationStatus.DIAJUKAN, ApplicationStatus.MENUNGGU_VERIFIKASI_BERKAS])
29     ).count()
30     perlu_survei_ppl = db.query(Application).filter(
31         Application.status.in_([ApplicationStatus.BERKAS_TERVERIFIKASI, ApplicationStatus.DITUGASKAN_KE_PPL,
32         ApplicationStatus.SURVEI_LAPANGAN])
33     ).count()
34     menunggu_persetujuan_akhir = db.query(Application).filter(
35         Application.status == ApplicationStatus.MENUNGGU_PERSETUJUAN_AKHIR
36     ).count()
37     disetujui = db.query(Application).filter(
38         Application.status.in_([ApplicationStatus.DISETUIJUI, ApplicationStatus.DIJADWALKAN_DISTRIBUSI, ApplicationStatus.
39         TERSALURKAN])
40     ).count()
41     ditolak = db.query(Application).filter(
42         Application.status.in_([ApplicationStatus.DITOLAK_BERKAS, ApplicationStatus.DITOLAK_LAPANGAN])
43     ).count()
44
45     total_petani = db.query(Farmer).count()
46     total_lahan_m2_res = db.query(func.sum(Land.luas_m2)).scalar() or Decimal("0")
47
48     return DashboardStatsAdmin(
49         total_pengajuan=total_pengajuan,
50         menunggu_verifikasi_berkas=menunggu_verifikasi_berkas,
51         perlu_survei_ppl=perlu_survei_ppl,
52         menunggu_persetujuan_akhir=menunggu_persetujuan_akhir,
53         disetujui=disetujui,
54         ditolak=ditolak,
55         total_petani=total_petani,
56         total_lahan_m2=total_lahan_m2_res,
57         kuota_urea_kg=Decimal("150000.00"),
58         kuota_npk_kg=Decimal("120000.00")
59     )
60
61 @router.get("/ppl-officers")
62 def get_ppl_officers(
63     current_user: User = Depends(require_role("ADMIN")),
64     db: Session = Depends(get_db)
65 ):
66     ppl_users = db.query(User).filter(User.role == "PPL", User.is_active == True).all()
67     result = []
68     for u in ppl_users:
69         active_surveys = db.query(Application).filter(
70             Application.assigned_ppl_id == u.id,
71             Application.status.in_([ApplicationStatus.DITUGASKAN_KE_PPL, ApplicationStatus.SURVEI_LAPANGAN])
72         ).count()
73         result.append({
74             "id": u.id,
75             "username": u.username,
76             "email": u.email,
77             "active_tasks": active_surveys,
78             "wilayah": "Mojokerto & Sekitarnya"
79         })

```

```

78     return result
79
80 @router.post("/applications/{id}/verify-docs")
81 def verify_application_docs(
82     id: int,
83     data: ApplicationAdminVerifyRequest,
84     current_user: User = Depends(require_role("ADMIN")),
85     db: Session = Depends(get_db)
86 ):
87     app = db.query(Application).filter(Application.id == id).first()
88     if not app:
89         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
90
91     app.admin_verifier_id = current_user.id
92     app.admin_verified_at = datetime.datetime.now()
93     app.catatan_admin_berkas = data.catatan
94
95     farmer_user_id = app.farmer.user_id if app.farmer else None
96
97     if data.action == "APPROVE":
98         app.status = ApplicationStatus.BERKAS_TERVERIFIKASI
99         if farmer_user_id:
100             db.add(Notification(
101                 user_id=farmer_user_id,
102                 judul="Berkas Terverifikasi",
103                 pesan=f"Berkas pengajuan #{app.id} telah lolos verifikasi admin. Menunggu penugasan survei PPL.",
104                 tipe="SUCCESS"
105             ))
106     elif data.action == "PERBAIKAN":
107         app.status = ApplicationStatus.PERLU_PERBAIKAN_BERKAS
108         if farmer_user_id:
109             db.add(Notification(
110                 user_id=farmer_user_id,
111                 judul="Perlu Perbaikan Berkas",
112                 pesan=f"Berkas pengajuan #{app.id} perlu diperbaiki: {data.catatan or 'Periksa kembali foto dokumen'}",
113                 tipe="WARNING"
114             ))
115     elif data.action == "TOLAK":
116         app.status = ApplicationStatus.DITOLAK_BERKAS
117         if farmer_user_id:
118             db.add(Notification(
119                 user_id=farmer_user_id,
120                 judul="Pengajuan Ditolak (Berkas)",
121                 pesan=f"Pengajuan #{app.id} ditolak pada verifikasi berkas. Alasan: {data.catatan}",
122                 tipe="DANGER"
123             ))
124     else:
125         raise HTTPException(status_code=400, detail="Aksi tidak valid (APPROVE / PERBAIKAN / TOLAK)")
126
127     db.commit()
128     db.refresh(app)
129
130     log_audit_action(
131         db,
132         user_id=current_user.id,
133         action=f"VERIFY_DOCS_{data.action}",
134         resource="applications",
135         resource_id=app.id,
136         details=f"Admin {current_user.username} verified docs: {data.action}. Note: {data.catatan}"
137     )
138
139     return {"message": "Status verifikasi berkas berhasil diperbarui", "status": app.status}
140
141 @router.post("/applications/{id}/assign-ppl")
142 def assign_ppl(
143     id: int,
144     data: ApplicationAssignPPLRequest,
145     current_user: User = Depends(require_role("ADMIN")),
146     db: Session = Depends(get_db)
147 ):
148     app = db.query(Application).filter(Application.id == id).first()
149     if not app:
150         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
151
152     ppl = db.query(User).filter(User.id == data.ppl_id, User.role == "PPL").first()
153     if not ppl:
154         raise HTTPException(status_code=400, detail="Petugas PPL tidak valid")
155
156     app.assigned_ppl_id = ppl.id
157     app.assigned_at = datetime.datetime.now()
158     app.catatan_penugasan = data.catatan
159     app.status = ApplicationStatus.DITUGASKAN_KE_PPL
160

```

```

161 # Notify PPL
162 db.add(Notification(
163     user_id=ppl.id,
164     judul="Tugas Survei Lapangan Baru",
165     pesan=f"Anda ditugaskan melakukan survei fisik lahan untuk pengajuan #{app.id} (Petani: {app.farmer.nama if app.farmer else 'Petani'}). Catatan: {data.catatan or '-'}",
166     tipe="INFO"
167 ))
168
169 # Notify Farmer
170 if app.farmer and app.farmer.user_id:
171     db.add(Notification(
172         user_id=app.farmer.user_id,
173         judul="PPL Telah Ditugaskan",
174         pesan=f"Petugas PPL ({ppl.username}) telah ditugaskan untuk melakukan verifikasi fisik lahan Anda.",
175         tipe="INFO"
176     ))
177
178 db.commit()
179 db.refresh(app)
180
181 log_audit_action(
182     db,
183     user_id=current_user.id,
184     action="ASSIGN_PPL",
185     resource="applications",
186     resource_id=app.id,
187     details=f"Assigned to PPL {ppl.username} (ID: {ppl.id})"
188 )
189
190 return {"message": "PPL berhasil ditugaskan", "status": app.status}
191
192 @router.post("/applications/{id}/final-approve")
193 def final_approve_application(
194     id: int,
195     data: ApplicationFinalApproveRequest,
196     current_user: User = Depends(require_role("ADMIN")),
197     db: Session = Depends(get_db)
198 ):
199     app = db.query(Application).filter(Application.id == id).first()
200     if not app:
201         raise HTTPException(status_code=404, detail="Pengajuan tidak ditemukan")
202
203     app.final_approver_id = current_user.id
204     app.final_approved_at = datetime.datetime.now()
205     app.catatan_final_admin = data.catatan
206
207     farmer_user_id = app.farmer.user_id if app.farmer else None
208
209     if data.action == "APPROVE":
210         app.status = ApplicationStatus.DISETUIJUI
211         app.jumlah_disetujui = data.jumlah_disetujui or app.jumlah_diajukan
212
213         # Keep distribution allocation separate from the physical bag QR.
214         existing_dist = db.query(Distribution).filter(Distribution.application_id == app.id).first()
215         if not existing_dist:
216             dist = Distribution(
217                 application_id=app.id,
218                 jumlah_disalurkan=app.jumlah_disetujui,
219                 status_penyaluran="MENUNGGU_PENGAMBILAN"
220             )
221             db.add(dist)
222             app.status = ApplicationStatus.DIJADWALKAN_DISTRIBUSI
223
224             if farmer_user_id:
225                 db.add(Notification(
226                     user_id=farmer_user_id,
227                     judul="Pengajuan Subsidi Disetujui!",
228                     pesan=f"Selamat! Pengajuan subsidi pupuk #{app.id} telah disetujui untuk kuota {app.jumlah_disetujui} kg.
229                     Pembukaan pupuk menggunakan QR pada karung pupuk.",
230                     tipe="SUCCESS"
231                 ))
232
233         elif data.action == "REJECT":
234             app.status = ApplicationStatus.DITOLAK_LAPANGAN
235             if farmer_user_id:
236                 db.add(Notification(
237                     user_id=farmer_user_id,
238                     judul="Pengajuan Subsidi Ditolak",
239                     pesan=f"Pengajuan subsidi pupuk #{app.id} ditolak pada keputusan akhir. Alasan: {data.catatan}",
240                     tipe="DANGER"
241                 ))
242         else:

```

```

242         raise HTTPException(status_code=400, detail="Aksi tidak valid (APPROVE / REJECT)")
243
244     db.commit()
245     db.refresh(app)
246
247     log_audit_action(
248         db,
249         user_id=current_user.id,
250         action=f"FINAL_APPROVE_{data.action}",
251         resource="applications",
252         resource_id=app.id,
253         details=f"Admin {current_user.username} decided: {data.action} ({app.jumlah_disetujui} kg)"
254     )
255
256     return {"message": "Keputusan akhir berhasil disimpan", "status": app.status}
257
258 @router.get("/audit-logs", response_model=List[AuditLogResponse])
259 def get_audit_logs(
260     limit: int = 50,
261     current_user: User = Depends(require_role("ADMIN")),
262     db: Session = Depends(get_db)
263 ):
264     return db.query(AuditLog).order_by(AuditLog.created_at.desc()).limit(limit).all()

```

Listing 7. backend/schema.sql

```

1  -- =====
2  -- Database Schema for E-Pupuk Kabupaten Mojokerto
3  -- =====
4  CREATE DATABASE IF NOT EXISTS epupuk CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
5  USE epupuk;
6
7  -- Drop in reverse foreign key order if needed
8  DROP TABLE IF EXISTS audit_logs;
9  DROP TABLE IF EXISTS notifications;
10 DROP TABLE IF EXISTS qr_scans;
11 DROP TABLE IF EXISTS distributions;
12 DROP TABLE IF EXISTS fertilizer_batches;
13 DROP TABLE IF EXISTS field_surveys;
14 DROP TABLE IF EXISTS applications;
15 DROP TABLE IF EXISTS lands;
16 DROP TABLE IF EXISTS fertilizers;
17 DROP TABLE IF EXISTS commodities;
18 DROP TABLE IF EXISTS farmers;
19 DROP TABLE IF EXISTS farmer_groups;
20 DROP TABLE IF EXISTS users;
21
22 -- 1. users
23 CREATE TABLE users (
24     id INT AUTO_INCREMENT PRIMARY KEY,
25     username VARCHAR(100) UNIQUE NOT NULL,
26     email VARCHAR(150) UNIQUE NOT NULL,
27     password_hash VARCHAR(255) NOT NULL,
28     role ENUM('PETANI', 'PPL', 'ADMIN') NOT NULL,
29     is_active BOOLEAN DEFAULT TRUE,
30     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
31 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
32
33 -- 2. farmer_groups (Kelompok Tani)
34 CREATE TABLE farmer_groups (
35     id INT AUTO_INCREMENT PRIMARY KEY,
36     nama_kelompok VARCHAR(150) NOT NULL,
37     ketua_id INT NULL,
38     wilayah VARCHAR(200) NOT NULL,
39     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
40 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
41
42 -- 3. farmers (Profil Petani)
43 CREATE TABLE farmers (
44     id INT AUTO_INCREMENT PRIMARY KEY,
45     user_id INT UNIQUE NOT NULL,
46     nama VARCHAR(150) NOT NULL,
47     nik VARCHAR(20) UNIQUE NOT NULL,
48     kontak VARCHAR(20),
49     alamat TEXT NOT NULL,
50     foto_ktp_url VARCHAR(500),
51     tanggal_registrasi DATE DEFAULT (CURRENT_DATE),
52     farmer_group_id INT NULL,
53     FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
54     FOREIGN KEY (farmer_group_id) REFERENCES farmer_groups(id) ON DELETE SET NULL
55 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
56
57 -- 4. commodities (Komoditas Pertanian)
58 CREATE TABLE commodities (
59     id INT AUTO_INCREMENT PRIMARY KEY,
60     nama_komoditas VARCHAR(100) NOT NULL,
61     standar_kebutuhan_kg_per_ha DECIMAL(10,2) NOT NULL
62 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
63
64 -- 5. fertilizers (Jenis Pupuk Bersubsidi)
65 CREATE TABLE fertilizers (
66     id INT AUTO_INCREMENT PRIMARY KEY,
67     nama_pupuk VARCHAR(100) NOT NULL,
68     satuan VARCHAR(20) DEFAULT 'kg'
69 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
70
71 -- 6. lands (Lahan Pertanian Petani)
72 CREATE TABLE lands (
73     id INT AUTO_INCREMENT PRIMARY KEY,
74     farmer_id INT NOT NULL,
75     lokasi_deskripsi VARCHAR(300),
76     alamat_lahan TEXT NOT NULL,
77     latitude DECIMAL(10,7),
78     longitude DECIMAL(10,7),
79     luas_m2 DECIMAL(12,2) NOT NULL,

```

```

80 commodity_id INT,
81 status_kepemilikan ENUM('MILIK','GARAP','SEWA') DEFAULT 'MILIK',
82 foto_lahan_url VARCHAR(500),
83 tanggal_registrasi DATE DEFAULT (CURRENT_DATE),
84 FOREIGN KEY (farmer_id) REFERENCES farmers(id) ON DELETE CASCADE,
85 FOREIGN KEY (commodity_id) REFERENCES commodities(id) ON DELETE SET NULL
86 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
87
88 -- 7. applications (Pengajuan Subsidi Pupuk)
89 CREATE TABLE applications (
90   id INT AUTO_INCREMENT PRIMARY KEY,
91   farmer_id INT NOT NULL,
92   land_id INT NOT NULL,
93   fertilizer_id INT NOT NULL,
94   jumlah_diajukan DECIMAL(10,2) NOT NULL,
95   kuota_maksimal DECIMAL(10,2),
96   status ENUM(
97     'DIAJUKAN',
98     'MENUNGGU_VERIFIKASI_BERKAS',
99     'PERLU_PERBAIKAN_BERKAS',
100    'BERKAS_TERVERIFIKASI',
101    'DITUGASKAN_KE_PPL',
102    'SURVEI_LAPANGAN',
103    'MENUNGGU_PERSETUJUAN_AKHIR',
104    'DISETUJUI',
105    'DITOLAK_BERKAS',
106    'DITOLAK_LAPANGAN',
107    'DIJADWALKAN_DISTRIBUSI',
108    'TERSALURKAN'
109 ) DEFAULT 'MENUNGGU_VERIFIKASI_BERKAS',
110 foto_ktp_snapshot_url VARCHAR(500),
111 foto_lahan_snapshot_url VARCHAR(500),
112 alamat_lahan TEXT,
113 latitude DECIMAL(10,7),
114 longitude DECIMAL(10,7),
115 tanggal_pengajuan TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
116 admin_verifier_id INT NULL,
117 admin_verified_at TIMESTAMP NULL,
118 catatan_admin_berkas TEXT,
119 assigned_ppl_id INT NULL,
120 assigned_at TIMESTAMP NULL,
121 catatan_penugasan TEXT,
122 final_approver_id INT NULL,
123 final_approved_at TIMESTAMP NULL,
124 jumlah_disetujui DECIMAL(10,2),
125 catatan_final_admin TEXT,
126 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
127 updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
128 FOREIGN KEY (farmer_id) REFERENCES farmers(id) ON DELETE CASCADE,
129 FOREIGN KEY (land_id) REFERENCES lands(id) ON DELETE CASCADE,
130 FOREIGN KEY (fertilizer_id) REFERENCES fertilizers(id),
131 FOREIGN KEY (admin_verifier_id) REFERENCES users(id) ON DELETE SET NULL,
132 FOREIGN KEY (assigned_ppl_id) REFERENCES users(id) ON DELETE SET NULL,
133 FOREIGN KEY (final_approver_id) REFERENCES users(id) ON DELETE SET NULL
134 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
135
136 -- 8. field_surveys (Survei Lapangan PPL)
137 CREATE TABLE field_surveys (
138   id INT AUTO_INCREMENT PRIMARY KEY,
139   application_id INT UNIQUE NOT NULL,
140   ppl_id INT NOT NULL,
141   tanggal_survei TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
142   kondisi_fisik_lahan ENUM('BAIK','CUKUP','TIDAK_LAYAK') NOT NULL,
143   keterangan_fisik_lahan TEXT,
144   kondisi_tanaman ENUM('SESUAI','TIDAK_SESUAI') NOT NULL,
145   keterangan_tanaman TEXT,
146   luas_lahan_aktual_m2 DECIMAL(12,2),
147   foto_survei_urls JSON,
148   catatan_ppl TEXT,
149   rekomendasi ENUM('SETUJU','TOLAK') NOT NULL,
150   FOREIGN KEY (application_id) REFERENCES applications(id) ON DELETE CASCADE,
151   FOREIGN KEY (ppl_id) REFERENCES users(id) ON DELETE CASCADE
152 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
153
154 -- 9. fertilizer_batches (Batch Pupuk)
155 CREATE TABLE fertilizer_batches (
156   id INT AUTO_INCREMENT PRIMARY KEY,
157   batch_code VARCHAR(50) UNIQUE NOT NULL,
158   fertilizer_id INT NOT NULL,
159   production_date DATE,
160   expired_date DATE NOT NULL,
161   stok_kg DECIMAL(12,2) DEFAULT 0,
162   FOREIGN KEY (fertilizer_id) REFERENCES fertilizers(id)

```

```

163 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
164
165 -- 10. distributions (Penyaluran dan QR Token)
166 CREATE TABLE distributions (
167   id INT AUTO_INCREMENT PRIMARY KEY,
168   application_id INT UNIQUE NOT NULL,
169   batch_id INT NULL,
170   jumlah_disalurkan DECIMAL(10,2) NOT NULL,
171   status_penyaluran ENUM('MENUNGGU_PENGAMBILAN','SELESAI','BATAL') DEFAULT 'MENUNGGU_PENGAMBILAN',
172   qr_code_hash VARCHAR(255) UNIQUE,
173   tanggal_penyaluran TIMESTAMP NULL,
174   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
175   FOREIGN KEY (application_id) REFERENCES applications(id) ON DELETE CASCADE,
176   FOREIGN KEY (batch_id) REFERENCES fertilizer_batches(id) ON DELETE SET NULL
177 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
178
179 -- 11. qr_scans (Catatan Pindai QR)
180 CREATE TABLE qr_scans (
181   id INT AUTO_INCREMENT PRIMARY KEY,
182   distribution_id INT NOT NULL,
183   scanner_user_id INT NOT NULL,
184   scan_timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
185   latitude DECIMAL(10,7),
186   longitude DECIMAL(10,7),
187   validation_status ENUM('VALID','EXPIRED','ALREADY_CLAIMED','INVALID') NOT NULL,
188   FOREIGN KEY (distribution_id) REFERENCES distributions(id) ON DELETE CASCADE,
189   FOREIGN KEY (scanner_user_id) REFERENCES users(id)
190 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
191
192 -- 12. notifications (Notifikasi Petani & PPL)
193 CREATE TABLE notifications (
194   id INT AUTO_INCREMENT PRIMARY KEY,
195   user_id INT NOT NULL,
196   judul VARCHAR(200) NOT NULL,
197   pesan TEXT NOT NULL,
198   tipe ENUM('INFO','WARNING','SUCCESS','DANGER') DEFAULT 'INFO',
199   is_read BOOLEAN DEFAULT FALSE,
200   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
201   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
202 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
203
204 -- 13. audit_logs (Audit Jejak Keamanan & Akses KTP)
205 CREATE TABLE audit_logs (
206   id INT AUTO_INCREMENT PRIMARY KEY,
207   user_id INT NULL,
208   action VARCHAR(100) NOT NULL,
209   resource VARCHAR(100) NOT NULL,
210   resource_id INT NULL,
211   details TEXT,
212   ip_address VARCHAR(45),
213   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
214   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
215 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

```

III. DESIGN SYSTEM

Design system E-Pupuk merancang visual konsisten, komponen antarmuka modular, skema warna yang harmonis, dan elemen navigasi intuitif untuk pengguna dari berbagai peran (Petani, PPL, Admin, dan Pimpinan Dinas).

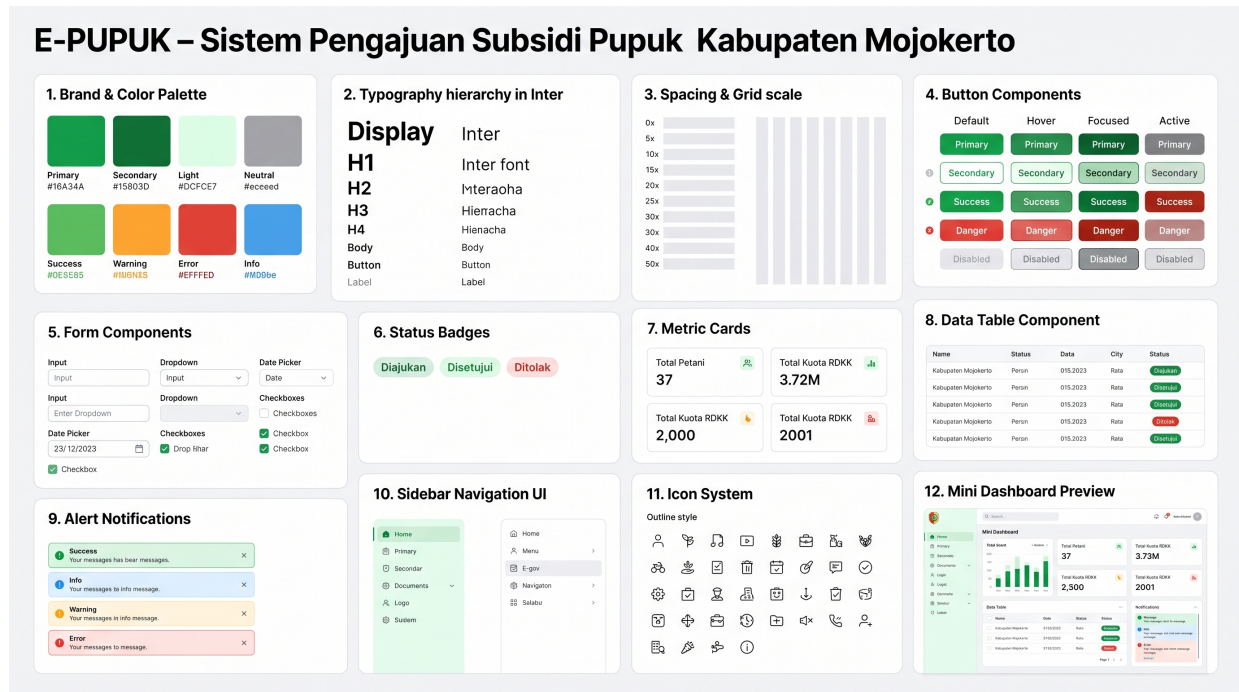


Fig. 1. Design System E-Pupuk

IV. ARSITEKTUR SISTEM

Arsitektur sistem E-Pupuk mengintegrasikan komponen frontend (React Web App), backend REST API (Python FastAPI), serta basis data terpusat (MySQL). Data terbuka pemerintah dimanfaatkan melalui modul data-seeding untuk memberikan informasi baseline kuota RDKK dan luas lahan.

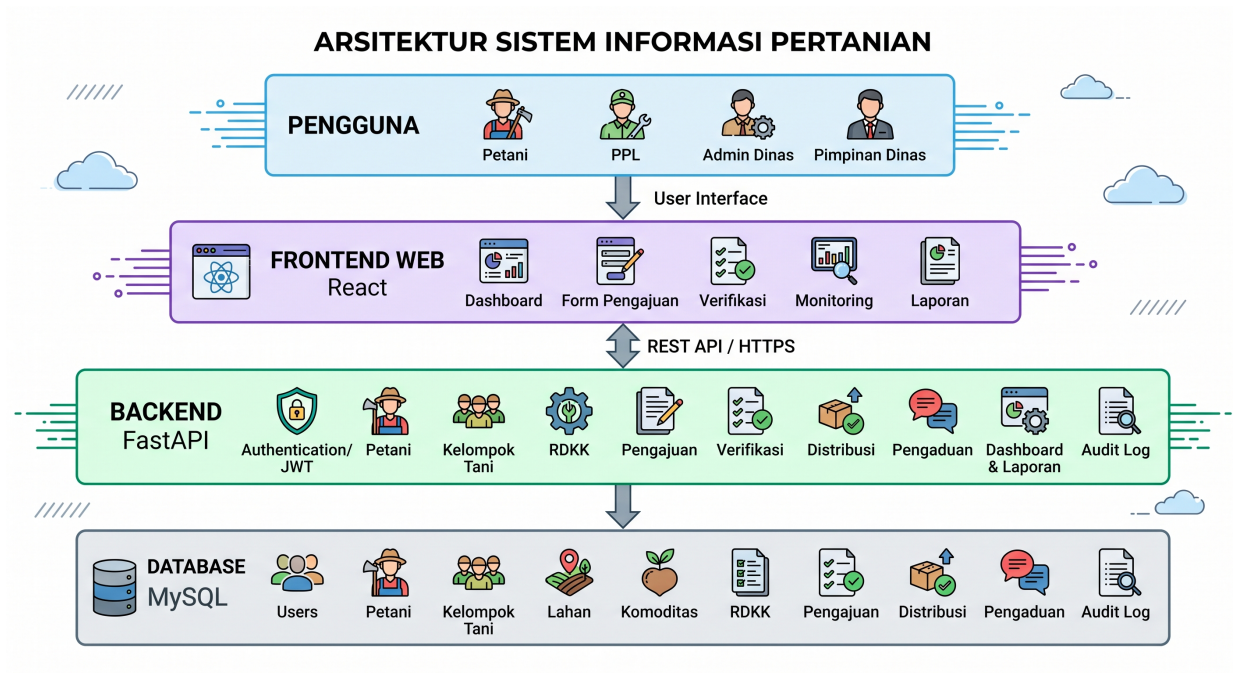


Fig. 2. Arsitektur Sistem E-Pupuk

V. MOCKUP UI

Bab ini menyajikan 5 rancangan antarmuka (*Mockup UI*) utama sistem E-Pupuk dari folder *Frontend* yang mencakup alur proses pengguna dari halaman utama, pengajuan petani, verifikasi PPL, hingga monitoring eksekutif Dinas Pertanian.

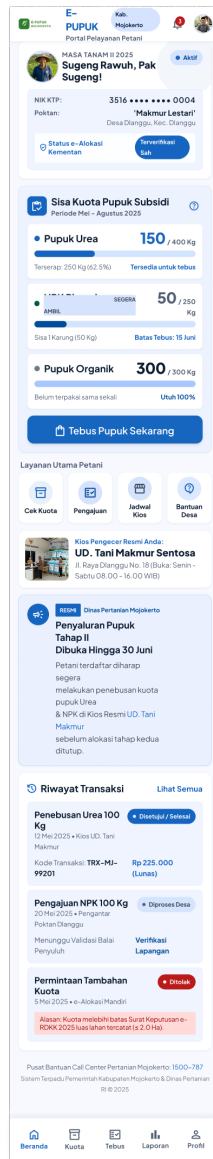


Fig. 3. Mockup UI – Halaman Utama / Beranda E-Pupuk Kabupaten Mojokerto



Fig. 4. Mockup UI – Dashboard Petani (Informasi Kuota RDKK dan Status Lahan)

Syarifatullah C.

Petani Poktan

←

Ajukan Kebutuhan Pupuk

MT-12025 • Poktan Tani Makmur

Petani Pemohon Terdaftar

Syarifatullah Ceva Efendy

Valid

Luas Lahan

0,5 Ha (5.000 m²)

Komoditas & Wilayah

Padi Sawah • Puri

Pilih Jenis Pupuk Bersubsidi

Wajib

Urea Bersubsidi

▼

HET Resmi Pemerintah:

Rp 2.250 / kg

Batas Kuota Lahan Anda: 125 kg

Tersedia untuk alokasi musim tanam saat ini sesuai RDKK Desa Puri

Jumlah Pengajuan (kg)

Kelipatan 5 kg

−

100

kg

+

Gunakan tombol atau ketik langsung

Status: Sesuai Kuota Lahan

Pengajuan dalam batas wajar sistem e-Alokasi Mojokerto

Estimasi Total Biaya Tebus

100 kg × Rp 2.250

Rp 225.000

Catatan Tambahan untuk PPL

Opsional

Pilih cepat kebutuhan pemupukan:

+ Pemupukan awal (Dasar)

+ Pemupukan susulan I

+ Pemupukan susulan II

+ Percepatan masa tanam

Tuliskan catatan khusus atau rencana tanggal tebus di Kios Puri...

Batal

Kirim Pengajuan Pupuk

+ Ajukan Pupuk

Beranda

Ajukan

Riwayat

Pengaduan

Profil

Fig. 5. Mockup UI – Form Pengajuan Kebutuhan Pupuk Subsidi oleh Petani

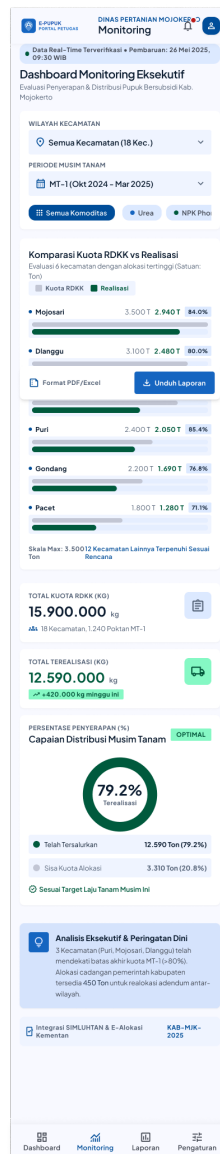


Fig. 7. Mockup UI – Dashboard Monitoring Eksekutif Dinas Pertanian